



PL-300

Search

CTRL

K



DAX-Zeitintelligenz

Enders Training

CONTENTS

Voraussetzung: Markierte Datumstabelle

Funktionsgruppe 1: Zusammenfassungen über einen Zeitraum (YTD, QTD, MTD)

DATESYTD, DATESQTD, DATESMTD

TOTALYTD, TOTALQTD, TOTALMTD

DATESBETWEEN und DATESINPERIOD

Funktionsgruppe 2: Vergleiche über den Zeitverlauf

SAMEPERIODLASTYEAR

Jahreswachstum (YoY %)

DATEADD und PARALLELPERIOD

Navigationsfunktionen

Weitere Zeitintelligenzberechnungen

Neue Vorkommen zählen (z. B. neue Kunden)

Momentaufnahmeberechnungen (Lagerbestand, Kontostände)

Zusammenfassung

Modul 3 · Skript 13

DAX-Zeitintelligenz

Datumsfilterkontexte ändern — YTD, Vorjahresvergleich, neue Kunden, Momentaufnahmen

Zeitintelligenz bezeichnet in DAX die Fähigkeit, Datumsfilter gezielt zu verschieben oder aufzuweiten — um z. B. den kumulierten Umsatz seit Jahresbeginn zu berechnen, den

Vorjahreswert gegenüberzustellen oder den Lagerbestand zum letzten Stichtag zu ermitteln. Im Kern geht es immer darum, den **Datumsfilterkontext zu ändern**, um zeitbezogene Fragen zu beantworten.

Voraussetzung: Markierte Datumstabelle

DAX-Zeitintelligenzfunktionen setzen zwingend voraus, dass das Modell mindestens eine als **Datumstabelle markierte** Tabelle enthält. Diese Tabelle muss eine Datumsspalte haben, die:

- **Eindeutige Werte** enthält
- **Volle Jahre** abdeckt (kein Jahresanfang oder -ende in der Mitte)
- **Keine BLANKs** enthält
- **Keine fehlenden Datumsangaben** aufweist (lückenlos)

Wie eine solche Datumstabelle per DAX oder Power Query erstellt und markiert wird, wurde in Skript 11 beschrieben.

Wichtig Die integrierten Zeitintelligenzfunktionen arbeiten mit Standardperioden: Jahren, Quartalen, Monaten. Für unregelmäßige Geschäftsperioden (z. B. 4-4-5-Kalender) oder Wochen-/Tagesanalysen müssen stattdessen **CALCULATE** mit eigenen Datumsfiltern verwendet werden.

Funktionsgruppe 1: Zusammenfassungen über einen Zeitraum (YTD, QTD, MTD)

Diese Funktionen verschieben den Filterkontext so, dass er einen kumulierten Zeitraum abdeckt — vom Beginn des Jahres/Quartals/Monats bis zum letzten Datum im aktuellen Filterkontext.

DATESYTD, DATESQTD, DATESMTD

Diese Funktionen geben eine einspaltige Tabelle mit Datumsangaben zurück und werden als Filterausdruck an **CALCULATE** übergeben:

Funktion	Liefert Datumsangaben von... bis...
DATESYTD	Jahresbeginn bis letztes Datum im Filterkontext
DATESQTD	Quartalsbeginn bis letztes Datum im Filterkontext
DATESMTD	Monatsbeginn bis letztes Datum im Filterkontext

TOTALYTD, TOTALQTD, TOTALMTD

Diese Funktionen kombinieren **CALCULATE** und **DATESYTD/DATESQTD/DATESMTD** in einem Schritt:

```
TOTALYTD(<Ausdruck>, <Datumsspalte>[, <Filter>][, <Jahresende>])
```

Das optionale **<Jahresende>**-Argument ist als Datum im Format **"MM-DD"** anzugeben — nur notwendig, wenn das Geschäftsjahr nicht am 31. Dezember endet.

```
-- Kumulierter Umsatz seit Geschäftsjahresbeginn (GJ endet am 30. Juni):
Revenue YTD =
TOTALYTD(
    [Revenue],
    'Date'[Date],
    "6-30"
)
```

Screenshot Dateiname: dax-matrix-revenue-ytd-activity-ssm.png — Matrixvisual mit Year und Month in den Zeilen sowie Revenue und Revenue YTD als Spalten. Die YTD-Werte steigen innerhalb jedes Jahres kumuliert an.

Hinweis **TOTALYTD** akzeptiert nur einen einzigen optionalen Filterausdruck. Sind mehrere Filter notwendig, **CALCULATE** mit **DATESYTD** als Filterargument verwenden.

DATESBETWEEN und DATESINPERIOD

Für frei definierte Zeiträume:

```
DATESBETWEEN(<Datumsspalte>, <Startdatum>, <Enddatum>)
DATESINPERIOD(<Datumsspalte>, <Startdatum>, <Anzahl>, <Intervall>)
```

DATESBETWEEN gibt alle Datumsangaben zwischen zwei festen Datumswerten zurück. Übergibt man BLANK als Startdatum, beginnt der Zeitraum beim ersten Datum in der Datumstabelle.

DATESINPERIOD gibt einen Zeitraum zurück, der an einem bestimmten Datum beginnt und sich über eine angegebene Anzahl von Intervallen erstreckt (z. B. 3 Monate, -1 Jahr).

Funktionsgruppe 2: Vergleiche über den Zeitverlauf

Diese Funktionen verschieben den Filterkontext um eine Zeitperiode — typisch für Vorjahresvergleiche und Wachstumsberechnungen.

SAMEPERIODLASTYEAR

Gibt dieselben Datumsangaben des Vorjahres zurück:

```
-- Umsatz des Vorjahres (gleicher Zeitraum, ein Jahr früher):  
Revenue PY =  
VAR RevenuePriorYear =  
    CALCULATE(  
        [Revenue],  
        SAMEPERIODLASTYEAR('Date'[Date])  
    )  
RETURN  
    RevenuePriorYear
```

Screenshot Dateiname: dax-matrix-revenue-py-ssm.png — Matrixvisual: Revenue PY für GJ2019 entspricht den Revenue-Werten von GJ2018 im gleichen Monat.

Jahreswachstum (YoY %)

Aufbauend auf dem Vorjahreswert lässt sich das prozentuale Wachstum berechnen — mit Variablen für Lesbarkeit und Leistung:

```
Revenue YoY % =  
VAR RevenuePriorYear =  
    CALCULATE(  
        [Revenue],  
        SAMEPERIODLASTYEAR('Date'[Date])  
    )  
RETURN  
    DIVIDE(  
        [Revenue] - RevenuePriorYear,
```

```
RevenuePriorYear  
)
```

Screenshot Dateiname: dax-matrix-revenue-yoy-ssm.png — Matrixvisual mit Revenue YoY % als prozentuale Veränderung gegenüber dem Vorjahresmonat.

DATEADD und PARALLELPERIOD

Für flexiblere Zeitverschiebungen:

```
DATEADD(<Datumsspalte>, <Anzahl>, <Intervall>)
```

DATEADD verschiebt die Datumsangaben um eine beliebige Anzahl von Intervallen (DAY, MONTH, QUARTER, YEAR) vor oder zurück. Negative Werte gehen zurück, positive vorwärts.

PARALLELPERIOD ähnelt **DATEADD**, gibt aber immer den vollständigen Zeitraum (ganzes Jahr/Quartal/Monat) zurück, statt denselben relativen Offset anzuwenden.

Navigationsfunktionen

Für einfache Vor-/Rückwärtsnavigation:

Funktion	Liefert
PREVIOUSDAY / NEXTDAY	Den vorherigen / nächsten Tag
PREVIOUSMONTH / NEXTMONTH	Den vorherigen / nächsten Monat (vollständig)
PREVIOUSQUARTER / NEXTQUARTER	Das vorherige / nächste Quartal
PREVIOUSYEAR / NEXTYEAR	Das vorherige / nächste Jahr

Weitere Zeitintelligenzberechnungen

Neue Vorkommen zählen (z. B. neue Kunden)

Ein häufiger Anwendungsfall: Zählen, wie viele Kunden in einem Zeitraum zum ersten Mal aktiv waren. Die Logik: **Kunden bis heute** minus **Kunden bis zum Vortag des Zeitraums**.

Schritt 1 — Kunden kumuliert bis Periodenende (LTD = Lifetime to Date):

```
Customers LTD =  
VAR CustomersLTD =  
    CALCULATE(  
        DISTINCTCOUNT(Sales[CustomerKey]),  
        DATESBETWEEN(  
            'Date'[Date],  
            BLANK(),  
            MAX('Date'[Date])  
        ),  
        'Sales Order'[Channel] = "Internet"  
    )  
RETURN  
    CustomersLTD
```

DATESBETWEEN mit BLANK als Startdatum beginnt beim allerersten Datum in der Datumstabelle. **MAX('Date'[Date])** gibt das letzte Datum im aktuellen Filterkontext zurück — also das Periodenende.

Screenshot Dateiname: dax-matrix-customers-ltd-ssm.png — Matrixvisual mit Customers LTD: kumulierte Anzahl eindeutiger Kunden, die von Beginn bis zum jeweiligen Monatsende aktiv waren.

Schritt 2 — Neue Kunden im Zeitraum:

```
New Customers =  
VAR CustomersLTD =  
    CALCULATE(  
        DISTINCTCOUNT(Sales[CustomerKey]),  
        DATESBETWEEN(  
            'Date'[Date],  
            BLANK(),  
            MAX('Date'[Date])  
        ),  
        'Sales Order'[Channel] = "Internet"  
    )  
VAR CustomersPrior =  
    CALCULATE(  
        DISTINCTCOUNT(Sales[CustomerKey]),  
        DATESBETWEEN(  
            'Date'[Date],  
            BLANK(),  
            MIN('Date'[Date]) - 1  
        ),  
        'Sales Order'[Channel] = "Internet"  
    )  
RETURN  
    CustomersLTD - CustomersPrior
```

`MIN('Date'[Date]) - 1` gibt den Tag vor dem ersten Datum im Filterkontext zurück — da Power BI Datumsangaben intern als Zahlen speichert, lässt sich durch Subtraktion von 1 ein Tag zurückgehen.

Screenshot Dateiname: dax-matrix-new-customers-ssm.png — Matrixvisual mit New Customers: Anzahl der Kunden, die in dem jeweiligen Monat erstmals aktiv waren.

Momentaufnahmeberechnungen (Lagerbestand, Kontostände)

Momentaufnahmedaten (z. B. tägliche Lagerbestände) haben eine besondere Eigenschaft: Sie dürfen **nicht über Zeit summiert** werden — die Summe der täglichen Lagerbestände ist keine sinnvolle Kennzahl. Korrekt ist es, den Bestand zum **letzten relevanten Datum** im Filterkontext zu ermitteln.

Lagerbestand am Monatsende mit LASTDATE:

```
Stock on Hand =  
CALCULATE(  
    SUM(Inventory[UnitsBalance]),  
    LASTDATE('Date'[Date])  
)
```

`LASTDATE` filtert auf das letzte Datum im Filterkontext. `SUM` ist notwendig, weil Measures nicht direkt auf Spalten verweisen können — da aber pro Produkt und Datum nur eine Zeile existiert, summiert `SUM` effektiv nur einen Wert.

Problem: Wenn für das letzte Datum kein Datensatz existiert (Wochenende, Feiertag, zukünftiges Datum), gibt das Measure BLANK zurück.

Lösung mit LASTNONBLANK:

```
Stock on Hand =  
CALCULATE(  
    SUM(Inventory[UnitsBalance]),  
    LASTNONBLANK(  
        'Date'[Date],  
        CALCULATE(SUM(Inventory[UnitsBalance]))  
    )  
)
```

LASTNONBLANK ist eine Iteratorfunktion: Sie durchläuft alle Datumsangaben im Filterkontext in **absteigender** chronologischer Reihenfolge und gibt das erste Datum zurück, für das der übergebene Ausdruck nicht BLANK ergibt. Das innere **CALCULATE** ist notwendig, um den Zeilenkontext der Iteration in einen Filterkontext zu überführen (Kontextübergang).

Hinweis Das Gegenstück zu **LASTNONBLANK** ist **FIRSTNONBLANK** — sie iteriert in aufsteigender chronologischer Reihenfolge. Beide Funktionen erfordern den Kontextübergang über **CALCULATE** im Ausdrucksargument.

Wichtig Momentaufnahme-Spalten wie **UnitsBalance** sollten im Modell **ausgeblendet** werden, damit Berichtsauctoren sie nicht versehentlich mit einer einfachen Summe in Visuals verwenden. Das Measure **Stock on Hand** ist die einzige korrekte Aggregation.

Screenshot Dateiname: dax-matrix-mountain-200-bike-stock-2020-june-ssm.png — Matrixvisual mit Stock on Hand per LASTNONBLANK: Nun werden auch Werte für Juni 2020 und die Jahresgesamtsumme korrekt angezeigt.

Zusammenfassung

Voraussetzung

Markierte Datumstabelle mit eindeutigen, lückenlosen Datumsangaben über volle Jahre. Ohne sie funktionieren keine Zeitintelligenzfunktionen. Für unregelmäßige Perioden CALCULATE mit eigenen Datumsfiltern nutzen.

YTD / QTD / MTD

TOTALYTD / TOTALQTD / TOTALMTD für kumulierte Werte. Jahresende bei abweichendem Geschäftsjahr als "MM-DD" angeben. DATESYTD usw. für die Verwendung in CALCULATE mit mehreren Filtern.

Vorjahresvergleich

SAMEPERIODLASTYEAR für Revenue PY und YoY %-Wachstum. DATEADD für beliebige Zeitverschiebungen. PARALLELPERIOD für vollständige Periodenvergleiche. Variablen für Lesbarkeit und Leistung nutzen.

Neue Vorkommen

DATESBETWEEN mit BLANK-Startdatum für LTD-Zählung. Neue Kunden = LTD – Kunden vor Periodenbeginn. MIN('Date'[Date]) - 1 für den Tag vor dem Periodenstart.

Momentaufnahmen

Lagerbestand und Kontostände nie über Zeit summieren. LASTDATE für den letzten Periodenstand. LASTNONBLANK wenn Lücken in den Daten möglich sind — iteriert rückwärts bis zum ersten Nicht-BLANK-Ergebnis.

Wichtige Funktionen im Überblick

TOTALYTD / DATESYTD · SAMEPERIODLASTYEAR · DATEADD · DATESBETWEEN ·
DATESINPERIOD · FIRSTDATE / LASTDATE · FIRSTNONBLANK / LASTNONBLANK ·
PREVIOUSYEAR / NEXTYEAR



Modul 3 – Semantisches Modell & DAX
Filterkontext & CALCULATE

Modul 3 – Semantisches Modell & DAX
Visuelle Berechnungen

